

Adaptive Robust Diagnostic Engine v2

Closing the Finite-Sample Robustness Gap: An Adaptive Diagnostic Engine for Simultaneous Leverage and Heteroscedasticity in Linear Regression

Author: Muhammed Kandeh (Matriculation: 22613547)

Institution: University of The Gambia, Department of Mathematics

Supervisor: Dr. I.A. Baba

Degree: Master of Science in Statistics and Data Science

Overview

The Adaptive Robust Diagnostic Engine is a four-stage statistical pipeline designed to address simultaneous high-leverage outliers and heteroscedasticity in linear regression:

1. **Stage 1:** Fast-MCD robust leverage detection with adaptive weight-floor selector
2. **Stage 2:** Weighted Least Absolute Deviation (WLAD) estimation via IRLS
3. **Stage 3:** LOWESS-Optimized Bootstrap Envelope Plot (LBEP) diagnostic visualization
4. **Stage 4:** Bias-Corrected and Accelerated (BCa) residual bootstrap inference

Key Innovation

Data-adaptive weight-floor selector * that extends practical robustness from ~15% (conventional fixed-floor) to over 35% contamination in finite samples.

Installation

```
git clone https://github.com/sirage896-blip/adaptive-robust-diagnostic-engine-v2.git
cd adaptive-robust-diagnostic-engine-v2
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Requirements

- Python 3.10+
- NumPy, SciPy, scikit-learn, statsmodels, matplotlib, pandas
- 6+ CPU cores recommended for parallelisation
- 16GB+ RAM for $n > 500$

Quick Start

```
from engine import AdaptiveRobustDiagnosticEngine
import pandas as pd
import numpy as np
```

```

# Load data
data = pd.read_csv('gambia_education/student_performance.csv')
X = data[['STR', 'PQT', 'SII', 'PSM']].values
y = data['performance_index'].values # Response variable

# Initialise and run engine
engine = AdaptiveRobustDiagnosticEngine(
    coverage=0.5,
    alpha=0.05,
    n_bootstrap=1000,
    lowess_frac=0.4, # LOWESS smoothing span (validated optimal, see Table B.3)
    random_state=42
)
results = engine.fit(X, y)

# Access results
print("Coefficients:", results['coefficients'])
print("95% BCa Confidence Intervals:", results['confidence_intervals'])
print("Outlier Indices:", results['outlier_indices'])
print("Adaptive Delta:", results['adaptive_delta'])

```

Module Structure

```

├─ engine.py                # Main pipeline orchestrator (Stages 1-4)
├─ mcd_core.py              # Fast-MCD and deterministic MCD implementations
│   ├── FastMCD class with C-step optimization
│   ├── Deterministic MCD with six initialization strategies
│   └─ Tikhonov regularization for near-singular covariance matrices
├─ adaptive_weights.py      # Adaptive weight-floor selection
│   ├── Computation of MCD eigenratio
│   └─ Closed-form  $\delta^*$  selector via Equation (3.12)
├─ wlad_solver.py           # WLAD estimation via IRLS
│   ├── Warm-starting with OLS estimates
│   ├── Convergence diagnostics
│   └─ Non-uniqueness handling
├─ diagnostics.py           # LBEP visualization
│   ├── Bootstrap percentile envelope computation
│   ├── LOWESS smoothing via statsmodels
│   └─ LBEP rendering with matplotlib
├─ bca_bootstrap.py         # BCa confidence interval construction
│   ├── Residual bootstrap with centered residuals
│   ├── Jackknife acceleration factor computation
│   └─ Bias-correction and percentile adjustment
├─ requirements.txt         # Exact package versions
└─ notebooks/              # Jupyter notebooks reproducing all figures/tables
    ├── simulation_studies.ipynb
    ├── benchmark_validation.ipynb
    └─ gambia_education.ipynb

```

Recommended Parameters

Parameter	Recommended Value	Notes
Bootstrap replications	B = 1000	B = 500 for exploratory analysis
LOWESS span	frac = 0.4	Best sensitivity/specificity balance (validated)
Clean subset threshold	w > 0.5	Standard threshold
Adaptive weight floor	* auto-selected	Via Equation (3.12)
IRLS convergence	10	Relative change in objective
IRLS max iterations	100	Sufficient for convergence
Ridge regularisation	= 10	Numerical stability

Computational Performance

Stage	n=100	n=500	n=1000
Fast-MCD + Adaptive Selector	0.48s	2.18s	5.41s
WLAD (IRLS)	0.15s	0.89s	2.10s
LBEP (B=1000)	12.4s	45.2s	98.6s
BCa Bootstrap (B=1000)	18.7s	52.3s	112.4s
Total (parallel, 6-core)	~12.5s	~22s	~48s

Note: The adaptive selector adds minimal computational overhead (<1% of total runtime).

Reproducibility

All Monte Carlo experiments use fixed random seeds (seed = 42). To reproduce:

```
# Simulation studies
python scripts/run_simulations.py

# Benchmark validation
python scripts/validate_benchmarks.py

# Gambian education analysis
python scripts/analyze_gambia.py
```

Or run the provided Jupyter notebooks in the `notebooks/` directory.

Datasets

- **Gambian Educational Data** (anonymised): data/gambia_education/
 - **Hawkins-Bradu-Kass (HBK)**: data/benchmarks/hbk.csv
 - **Brownlee Stack Loss**: data/benchmarks/brownlee.csv
-

Citation

If you use this engine in your research, please cite:

```
@mastersthesis{kandeh2027,  
  author = {Kandeh, Muhammed},  
  title  = {Closing the Finite-Sample Robustness Gap:  
           An Adaptive Diagnostic Engine for Simultaneous Leverage  
           and Heteroscedasticity in Linear Regression},  
  school = {University of The Gambia},  
  year   = {2027},  
  type   = {Master's Thesis},  
  department = {Department of Mathematics}  
}
```

License

Released under the [MIT License](#).

Contact

For questions about methodology or implementation:

- **Open an issue** on the [GitHub repository](#)
 - **Email:** mkandeh@utg.edu.gm (academic inquiries)
-

Acknowledgements

- Supervisor: Dr. I.A. Baba
- Ministry of Basic and Secondary Education (MoBSE), The Gambia
- World Bank and UNESCO for international education data resources